



LECTURE

Java Spring Boot

From zero to a running REST API — fast.

**I**

The Landscape

Modern apps, opinionated frameworks

II

Case Study

A Car Rental API

III

REST & CRUD

The language your API speaks

IV

Architecture

The four layers of Spring Boot

V

How It Flows

Request flow and MVC

VI

JPA Deep Dive

Objects ↔ database tables

VII

Code Walkthrough

Entity · Repository · Service · Controller

VIII

Comparison & Wrap-up

Traditional Java vs Spring Boot



By the end of this lecture, you will...



Understand what Spring Boot is

1

and why it changed the Java platform.



Read a REST API contract

2

recognize endpoints, verbs, status codes, and JSON payloads.



Name the four architectural layers

3

and explain what lives in each one.



Read a Spring Boot codebase

4

and identify Entity, Repository, Service, Controller.



Explain common annotations

5

@Entity, @Service, @RestController, @Autowired and more.



Compare approaches

6

articulate where Spring Boot saves effort vs traditional Java.



Anatomy of a modern web app



Front-end Framework

Renders UI, handles user interactions — React, Angular, Vue.js.



Back-end Framework

Business logic, requests, database — Spring Boot, Express, Django.



Database

Stores and retrieves data — PostgreSQL, MySQL, MongoDB, Redis.



APIs

How components and external systems communicate.



Authentication & Authorization

Who signs in, and what they can do.



Caching

Frequently-used data held in fast memory — Redis, Memcached.



Monitoring & Logging

Performance, usage, and error tracking in production.



Seven pieces

Every modern app assembles these building blocks.

Next slide →

Where does Spring Boot fit?



Where Spring Boot fits in the stack



Today: we zoom in on the middle layer — the back-end API.



What is Spring Boot?

“

Before Spring Boot, it wasn't uncommon to spend days wiring an MVC framework to a persistence framework just to talk to a database — before any real coding could begin.

Spring Boot changed that.



An opinionated view

Best-practice defaults already decided — less bikeshedding, more building.



Build apps very quickly

Productivity comparable to Ruby on Rails, Django, or Flask.



Standalone Java apps

No external app server: Tomcat is embedded inside the JAR.



Just run the JAR

A single self-contained file you run with the plain `java` command.



What does “opinionated” actually mean?

*An **opinionated framework** ships with predefined conventions, best practices, and design patterns. The framework decides **how things should be done** — and you follow that structure.*

Ruby on Rails

Strict conventions for web apps

Angular

Comprehensive SPA framework

Spring Boot

Opinionated Java for back-ends

← today's topic

Same family — different languages, same philosophy: convention over configuration.



Opinionated: benefits and trade-offs



Benefits

- Ease of use and consistency across projects
- Simplicity prioritized over endless customization
- Predefined structure — great for newcomers
- Less time on architectural decisions
- Code familiar to any team member



Trade-offs

- Conventions may not fit every use case
- Customization can feel constrained
- You must learn “the framework way”
- Breaking convention usually hurts more than helps
- Under-the-hood complexity can surprise you

Most teams find the benefits dominate — especially at project start and when onboarding new developers.

C H E C K P O I N T

End of Part I · The Landscape



In your own words — what does it mean for a framework to be “opinionated”?



Think about: what choices has the framework made for you? What do you gain, and what do you give up?



Building a Car Rental API



THE SCENARIO

A car rental company needs a back-end API

*Customers search, book, and manage rentals.
Staff manage the fleet.*



Why this example?

1

Concrete and relatable

Everyone has rented (or imagined renting) a car.

2

Rich in CRUD

Create bookings, read listings, update details, delete rentals.

3

Real constraints

Dates, locations, availability — genuine business logic.

4

Scales naturally

From a tiny prototype to a fleet management platform.



The 8 endpoints we need

GET

Search available cars*by location, date and time*

POST

Reserve a car*pickup/drop-off + payment*

DELETE

Cancel a reservation*frees the car for others*

GET

Get rental history*user's past bookings*

PUT

Update rental info*change dates or locations*

GET

Get available locations*pickup & drop-off sites*

GET

Get pricing information*period, location, car type*

POST/PUT/DELETE

Manage car inventory*add / update / remove cars*

We'll use this scenario to compare traditional Java and Spring Boot throughout the lecture.



What we're comparing

Traditional Java

- Manual wiring
- Verbose configuration files
- External app server required
- More boilerplate code
- Slower to get to first running endpoint

VS

Java Spring Boot

- Auto-configuration
- Sensible defaults everywhere
- Embedded Tomcat — runs standalone
- Minimal boilerplate
- Working endpoint in minutes

We'll measure across four dimensions: development process · time · complexity · maintainability.



CRUD — the four verbs your API speaks

C Create Add new data HTTP POST EXAMPLE <i>Create a new booking</i>	R Read Retrieve data HTTP GET EXAMPLE <i>View booking details</i>	U Update Modify data HTTP PUT / PATCH EXAMPLE <i>Change pickup dates</i>	D Delete Remove data HTTP DELETE EXAMPLE <i>Cancel a booking</i>
---	---	--	--

Takeaway: *CRUD operations are implemented via **RESTful** API patterns.*



REST — the five principles

REST

= Representational State Transfer — an architectural style for networked APIs.

1

Stateless

Every request carries all the info it needs

2

Client–Server

Clean separation of concerns

3

Cacheable

Responses can be cached for performance

4

Layered System

Multiple layers, each with one job

5

Code on Demand

Optional — server may send executable code



Resources & URIs — naming things



A resource

...is a “thing” in your system: a car, a user, a booking, a location.



A URI

...is the unique address for that resource. Collection vs single item.

Convention:

plural nouns for collections
singular ID for specific items

URI examples — Car Rental API

GET	/cars	# list all cars
GET	/cars/{id}	# one specific car
POST	/cars	# create a new car
PUT	/cars/{id}	# update that car
DELETE	/cars/{id}	# remove that car
GET	/users/{id}/bookings	# bookings for one user



HTTP methods ↔ CRUD

GET	→	Read Retrieve a resource — safe and idempotent (no side effects)
POST	→	Create Add a new resource to the server
PUT	→	Update Replace an entire resource with new data
PATCH	→	Update Partial update — change only specific fields
DELETE	→	Delete Remove a resource from the server

Memorize this table — you'll see these four verbs everywhere in back-end development.



HTTP status codes — the server's reply

2xx Success <i>It worked</i>	3xx Redirection <i>Look somewhere else</i>	4xx Client error <i>Your fault</i>	5xx Server error <i>Our fault</i>
--	--	--	---

200	OK	<i>Request succeeded — most common success</i>
201	Created	<i>Resource was created (great for POST)</i>
204	No Content	<i>Success, nothing to send back (great for DELETE)</i>
400	Bad Request	<i>Malformed input or missing data</i>
401	Unauthorized	<i>You need to log in</i>
404	Not Found	<i>That resource does not exist</i>
500	Internal Server Error	<i>Something broke on our side</i>



JSON — the common data format

{ }

JavaScript Object Notation

- ✓ **Human-readable** *You can actually read it*
- ✓ **Language-agnostic** *Every language parses it*
- ✓ **Lightweight** *Small on the wire*
- ✓ **Hierarchical** *Nested objects & arrays*

car.json

```
{  
  "id": 42,  
  "model": "Tesla Model 3",  
  "year": 2024,  
  "available": true,  
  "pickup_locations": [  
    "Skopje Airport",  
    "City Center"  
  ],  
  "price_per_day": 49.99  
}
```



Creating a car — request in, response out

1. Client sends request

```
POST /cars HTTP/1.1
Host: api.carrental.com
Content-Type: application/json

{
  "model": "VW Golf",
  "year": 2023,
  "price_per_day": 35.00
}
```



2. Server replies

```
HTTP/1.1 201 Created
Location: /cars/42

{
  "id": 42,
  "model": "VW Golf",
  "year": 2023,
  "price_per_day": 35.00,
  "available": true
}
```

C H E C K P O I N T

End of Part III · REST & CRUD



Which HTTP method would you use to cancel a booking — and why?



Think CRUD. Cancelling = removing. Status 204 No Content is often the right response.



Batteries included — what Spring Boot embeds

Embedded servlet container — Spring Boot includes **Tomcat, Jetty, or Undertow** inside your app so it runs without any external server.



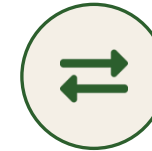
Web Server

Static files, dynamic content, RESTful services.



Database

MySQL, PostgreSQL, MongoDB — ready to connect.



Message Broker

Async messaging via Apache Kafka or RabbitMQ.



Caching

Ehcache or Hazelcast to reduce database load.



Security

OAuth2, Spring Security, Apache Shiro.

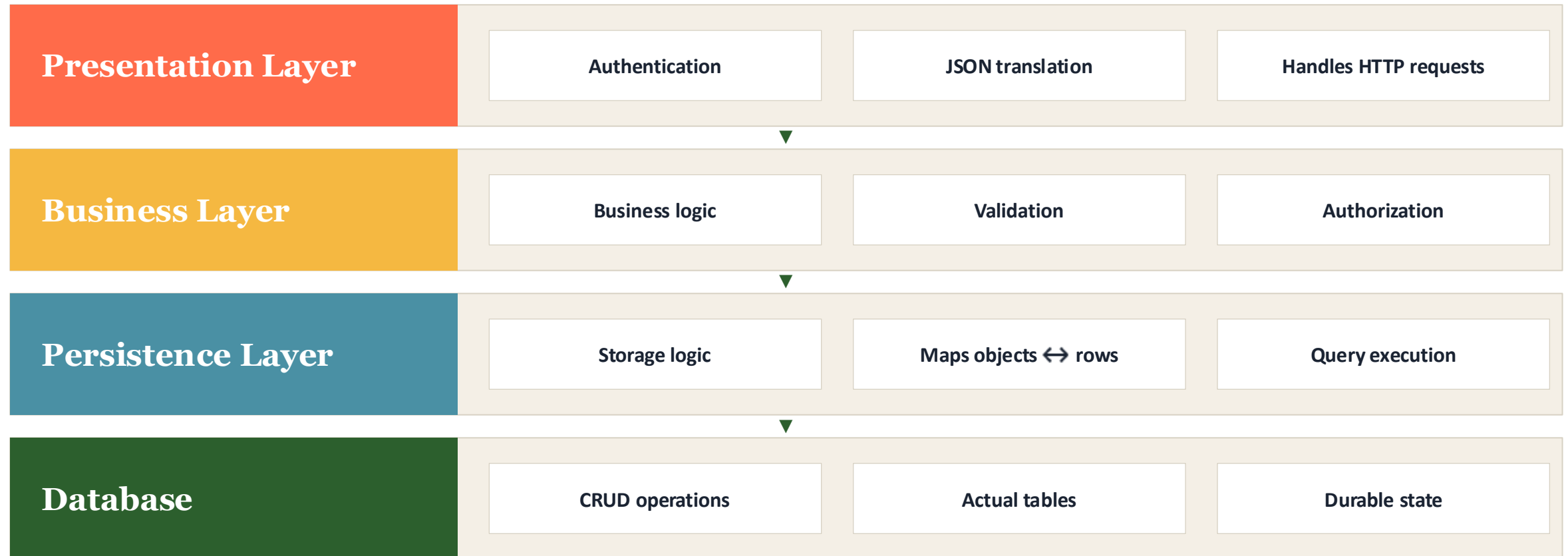


Monitoring

Spring Boot Actuator exposes health and metrics.



The four layers of Spring Boot



Next: a slide per layer with concrete examples.



Layer 1: Presentation Layer

Presentation Layer

RESPONSIBILITIES

- Receives incoming HTTP requests
- Translates JSON ↔ Java objects
- Authenticates requests
- Passes work down to the Business Layer
- Sends HTTP response back to the client



Concrete example

When a mobile app sends `POST /cars` with a JSON body, this layer is the first to receive it — it parses the JSON, checks the caller is logged in, and calls the service.

IN SPRING BOOT

```
@RestController classes  
@PostMapping, @GetMapping  
@RequestBody, @PathVariable
```



Layer 2: Business Layer

Business Layer

RESPONSIBILITIES

- Holds the business logic
- Validates input beyond basic syntax
- Enforces authorization rules
- Orchestrates calls to the persistence layer
- Combines data from multiple sources



Concrete example

Before saving a new booking, this layer checks: is the car actually available on those dates? Is the user allowed to book this category? Is the price computed correctly?

IN SPRING BOOT

```
@Service classes  
Domain rules live here  
Not in the controller, not in the repository
```



Layer 3: Persistence Layer

Persistence Layer

RESPONSIBILITIES

- Stores and retrieves data
- Translates Java objects ↔ database rows
- Runs SQL queries (often generated for you)
- Manages transactions
- Shields the rest of the app from SQL specifics



Concrete example

To find all cars available on a given date, this layer turns a method call like `findAvailableBetween(from, to)` into an actual SQL query, executes it, and returns Java objects.

IN SPRING BOOT

```
@Repository interfaces  
JpaRepository<Entity, ID>  
Query methods by name
```



Layer 4: Database

Database

RESPONSIBILITIES

- The actual storage engine
- Runs SQL (or NoSQL queries) on your behalf
- Enforces schema, types, and constraints
- Provides durability — data survives restarts
- Lives outside your JVM, usually on another machine



Concrete example

For the car rental API this is a PostgreSQL instance with tables like `cars`, `users`, `bookings`. Indexes and foreign keys live here, ensuring the data stays consistent over time.

IN SPRING BOOT

Not your Java code —
PostgreSQL, MySQL, MongoDB
Configured in `application.properties`



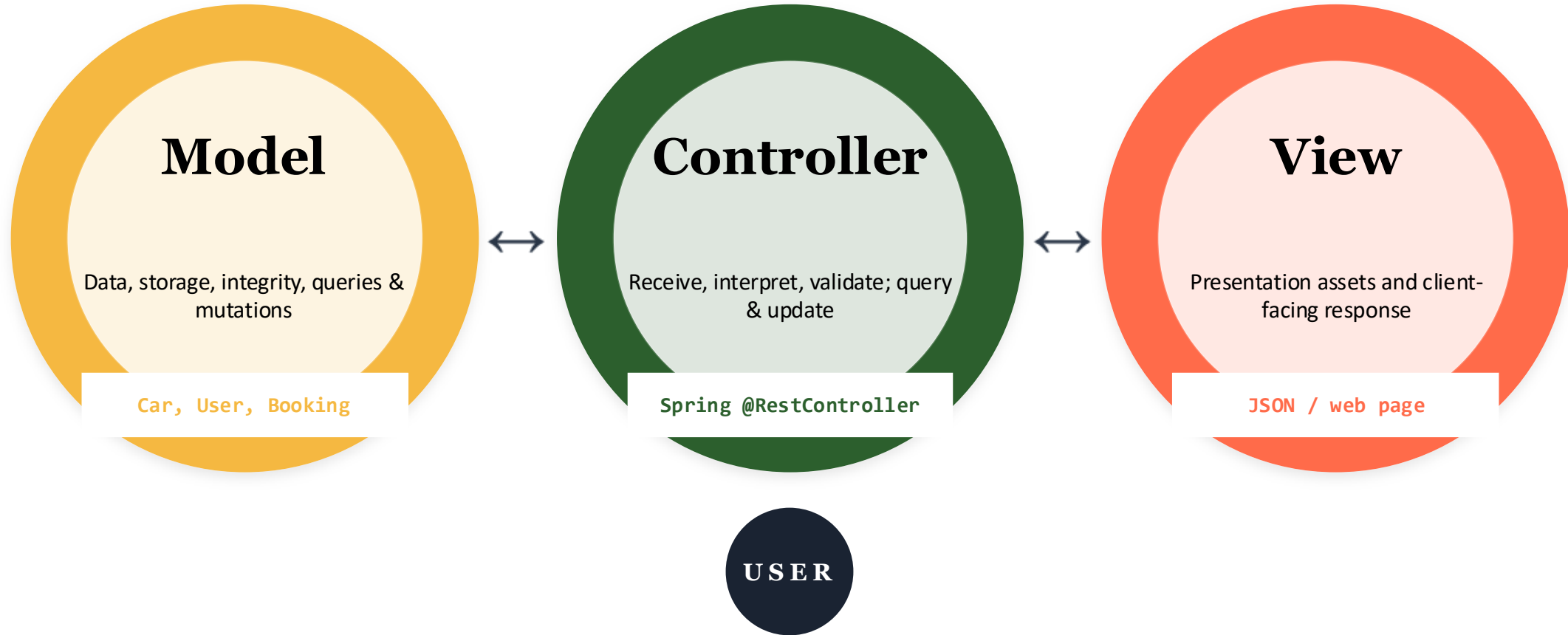
One HTTP request, start to finish



- 1 Client sends an HTTP request (e.g. GET /cars/42).
- 2 Controller maps the URL and delegates to the Service.
- 3 Service runs business logic — validation, authorization, composition.
- 4 Repository performs CRUD against the database via JPA.
- 5 Response flows back up the chain — usually as JSON.



MVC — Model · View · Controller



MVC is the pattern — Spring Boot is the implementation.



Why MVC matters



Separation of concerns

Each component has a clear and distinct responsibility.



Maintainability

Update, test, and debug pieces independently.



Reusability

Components can be reused across features or projects.



Scalability

Easier to scale and modify the app as it grows.

You'll meet MVC again and again

Ruby on Rails · Django · ASP.NET · Laravel · Angular · and many more — all use the MVC pattern.

Learn it once, recognize it everywhere.



JPA — Java objects meet database tables

JPA (Java Persistence API) is the standard API for **Object–Relational Mapping** in Java: work with objects, not SQL strings.



Java world

```
Car car = new Car();  
car.setModel("Tesla");  
car.setYear(2024);  
repo.save(car);
```



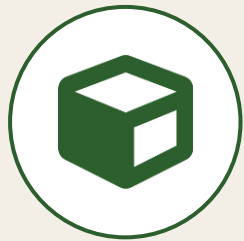
Database world

id	model	year
42	Tesla	2024
43	VW Golf	2023
44	Mazda 3	2024

You write Java. JPA handles the SQL.



The four pieces of JPA



Entities

Java classes that represent tables.

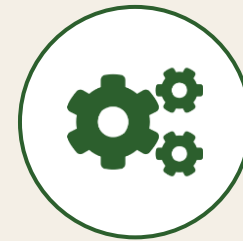
```
@Entity @Table @Id @Column
```



JPA Repositories

Interfaces that expose CRUD operations.

```
CarRepository, UserRepository
```



Entity Managers

Manages entities & transactions behind the scenes.

```
persist · merge · remove · find
```



Persistence Context

Runtime home for entity instances — caching, lazy loading.

```
managed by the framework
```



Spring annotations you'll see everywhere

Web / Controller

@RestController

Marks a class as a REST API controller

@RequestMapping

Base URL for the controller, e.g. "/cars"

@GetMapping

Handle HTTP GET requests

@PostMapping

Handle HTTP POST requests

@PathVariable

Extract a value from the URL path

@RequestBody

Parse JSON body into a Java object

Service / Wiring

@Service

Marks a class as business-logic layer

@Autowired

Inject a dependency automatically

@Component

Generic Spring-managed bean

Persistence / JPA

@Entity

Marks a class as a database table

@Table

Override the default table name

@Id

Marks the primary-key field

@GeneratedValue

Auto-generate the ID

@Column

Customize column mapping

@Repository

Marks a class as a data-access layer



Step 1 — the PostgreSQL table

Starting point

Every Spring Boot project that talks to a database begins with a **schema**.

For the rest of this walkthrough we'll use a simple **employee** table with three columns.

Why “employee” and not “car”?

Smaller schema = focus on Spring Boot, not on domain modelling.

schema.sql

```
CREATE TABLE employee (  
  id      SERIAL PRIMARY KEY,  
  name    VARCHAR(255),  
  position VARCHAR(255)  
);
```



Step 2 — Spring Boot configuration

application.properties

One file tells Spring Boot:

- Where the database is
- What credentials to use
- Which SQL dialect to speak
- Whether to auto-create tables
- Whether to log SQL

src/main/resources/application.properties

```
# DataSource settings
spring.datasource.url=jdbc:postgresql://localhost:5432/employee_db
spring.datasource.username=your_username
spring.datasource.password=your_password

# JPA / Hibernate settings
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```



Step 3 — the Entity class

Employee.java

```
import javax.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String name;
    private String position;

    // constructors, getters, setters
}
```

What it does

Represents one row of the **employee** table.

Each field = one column.

JPA reads the annotations to know how to store and load it.



Step 3b — what those annotations mean

@Entity

Tells JPA: this class maps to a database table.

By default, table name = class name.
Override with `@Table(name="...")`.

@Id

Marks the primary-key field.

Every entity must have exactly one `@Id`.

@GeneratedValue

The database should generate the ID automatically.

strategy = AUTO | IDENTITY | SEQUENCE | TABLE.

@Column

Customize the column mapping (optional).

e.g. `@Column(name="full_name", nullable=false, length=120)`



Step 4 — the Repository interface

EmployeeRepository.java

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {

    // custom query methods can go here

}
```

You get these methods — for free:

<code>save(entity)</code> insert or update	<code>findById(id)</code> Load one row
<code>findAll()</code> Load all rows	<code>deleteById(id)</code> remove a row
<code>count()</code> total row count	<code>existsById(id)</code> check presence

Interface, not class

You **declare** what you need.

Spring Data JPA generates the implementation at runtime.

No SQL. No boilerplate.

This is the feature that really makes Spring Boot feel magical.



Step 5 — the Service class

EmployeeService.java

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repo;

    public Employee addEmployee(Employee employee) {
        return repo.save(employee);
    }

    public Employee updateEmployee(Long id, Employee details) {
        Employee e = repo.findById(id)
            .orElseThrow(() -> new RuntimeException("not found"));
        e.setName(details.getName());
        e.setPosition(details.getPosition());
        return repo.save(e);
    }
}
```

Key points

@Service

Marks this as business-logic layer.

@Autowired

Spring injects the repo automatically.

findById

Returns Optional — force a throw if missing.

save()

Same method for insert and update.



Step 6 — the Controller class

EmployeeController.java

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @Autowired
    private EmployeeService service;

    @PostMapping
    public Employee add(@RequestBody Employee e) {
        return service.addEmployee(e);
    }

    @PutMapping("/{id}")
    public Employee update(
        @PathVariable Long id,
        @RequestBody Employee e) {
        return service.updateEmployee(id, e);
    }
}
```

The full stack

@RestController

is the entry point

↓ *calls*

@Service

for business logic

↓ *calls*

@Repository

for data access

↓ *touches*

@Entity → Database

the storage



Traditional Java vs. Spring Boot

AREA	Traditional Java	Spring Boot
Development Process	 Longer setup, manual configuration, lots of boilerplate.	 Faster setup, auto-configuration, minimal boilerplate.
Database Interaction	 Manual connection handling — more error-prone.	 Spring Data JPA — clean, declarative, maintainable.
API Design	 Hand-rolled REST patterns, error handling, validation.	 Spring MVC gives REST + error handling + validation.
Testing & Debugging	 Manual setup; more time fixing errors.	 Spring Boot Test simplifies testing and debugging.



Performance · Maintainability · Scalability

 Performance	Traditional Lower performance due to manual configuration and management.	Spring Boot Optimized with intelligent defaults and auto-configuration.
 Maintainability	Traditional Complex codebase, harder to maintain as the app grows.	Spring Boot Modular and cleaner codebase — easier to maintain and scale.
 Scalability	Traditional Scaling requires substantial refactoring.	Spring Boot Cleaner layers mean easier growth — just add more instances.

Spring Boot doesn't just save time — it produces a codebase that stays healthier over years.



Why Spring Boot — the five wins



Faster development

01

Opinionated defaults mean less setup, more building.



Less boilerplate code

02

Annotations and Spring Data remove repetitive plumbing.



Easier database interaction

03

JPA + Spring Data give you CRUD almost for free.



Improved maintainability

04

Clean layers — Controller · Service · Repository.



Enhanced API design

05

Built-in REST patterns, validation, and error handling.

The big picture

Spring Boot lets you focus on what your app does — instead of how to start it.

END OF LECTURE



Thank you

Questions & discussion



Where to next

Try Spring Initializr · clone the employee example · add your own @Entity · explore Spring Boot Actuator.